

Ramulator: A Brief Introduction

Presenter:
Nisa Bostancı
Prof. Onur Mutlu
XiangShan Tutorial
@ MICRO'25

SAFARI

ETH zürich

Ramulator

A Fast and Extensible DRAM Simulator
[IEEE Computer Architecture Letters'15]

Ramulator Motivation

- DRAM and Memory Controller landscape is changing
- Many new and upcoming standards
- Many new controller designs
- A **fast** and **easy-to-extend** simulator is very much needed

<i>Segment</i>	<i>DRAM Standards & Architectures</i>
Commodity	DDR3 (2007) [14]; DDR4 (2012) [18]
Low-Power	LPDDR3 (2012) [17]; LPDDR4 (2014) [20]
Graphics	GDDR5 (2009) [15]
Performance	eDRAM [28], [32]; RLD RAM3 (2011) [29]
3D-Stacked	WIO (2011) [16]; WIO2 (2014) [21]; MCDRAM (2015) [13]; HBM (2013) [19]; HMC1.0 (2013) [10]; HMC1.1 (2014) [11]
Academic	SBA/SSA (2010) [38]; Staged Reads (2012) [8]; RAIDR (2012) [27]; SALP (2012) [24]; TL-DRAM (2013) [26]; RowClone (2013) [37]; Half-DRAM (2014) [39]; Row-Buffer Decoupling (2014) [33]; SARP (2014) [6]; AL-DRAM (2015) [25]

Table 1. Landscape of DRAM-based memory

Ramulator

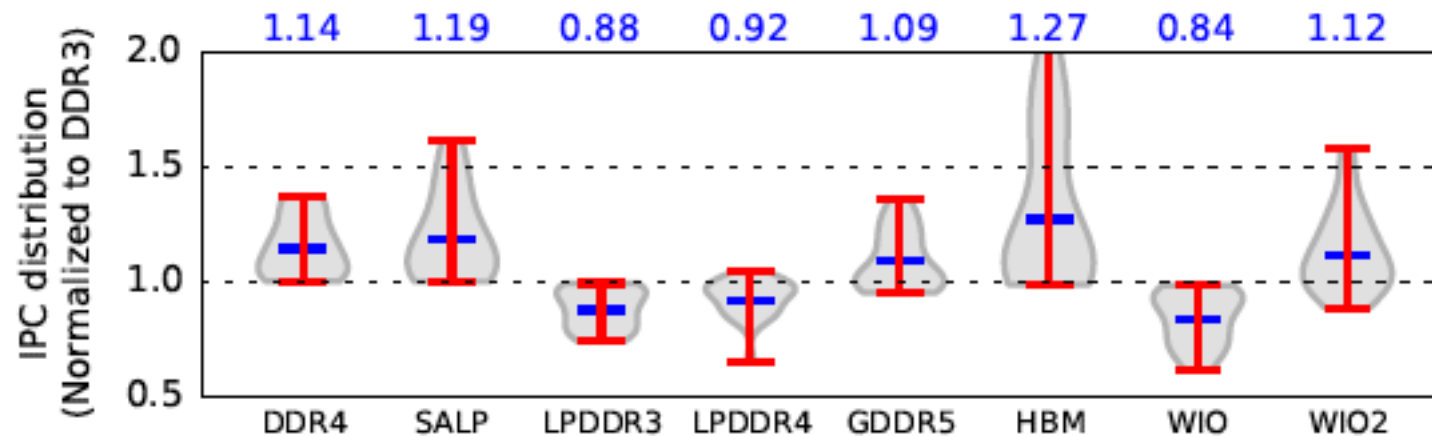
- Provides out-of-the box support for many DRAM standards:
 - DDR3/4, LPDDR3/4, GDDR5, WIO1/2, HBM, PCM, STT-RAM
 - new proposals (SALP, AL-DRAM, TLDRAM, RowClone, and SARP)
- ~2.5X faster than fastest open-source simulator
- Modular and extensible to different standards

<i>Simulator</i> <i>(clang -O3)</i>	<i>Cycles (10⁶)</i>		<i>Runtime (sec.)</i>		<i>Req/sec (10³)</i>		<i>Memory</i> <i>(MB)</i>
	<i>Random</i>	<i>Stream</i>	<i>Random</i>	<i>Stream</i>	<i>Random</i>	<i>Stream</i>	
Ramulator	652	411	752	249	133	402	2.1
DRAMSim2	645	413	2,030	876	49	114	1.2
USIMM	661	409	1,880	750	53	133	4.5
DrSim	647	406	18,109	12,984	6	8	1.6
NVMain	666	413	6,881	5,023	15	20	4,230.0

Table 3. Comparison of five simulators using two traces

Case Study: Comparison of DRAM Standards

<i>Standard</i>	<i>Rate (MT/s)</i>	<i>Timing (CL-RCD-RP)</i>	<i>Data-Bus (Width×Chan.)</i>	<i>Rank-per-Chan</i>	<i>BW (GB/s)</i>
DDR3	1,600	11-11-11	64-bit × 1	1	11.9
DDR4	2,400	16-16-16	64-bit × 1	1	17.9
SALP [†]	1,600	11-11-11	64-bit × 1	1	11.9
LPDDR3	1,600	12-15-15	64-bit × 1	1	11.9
LPDDR4	2,400	22-22-22	32-bit × 2*	1	17.9
GDDR5 [12]	6,000	18-18-18	64-bit × 1	1	44.7
HBM	1,000	7-7-7	128-bit × 8*	1	119.2
WIO	266	7-7-7	128-bit × 4*	1	15.9
WIO2	1,066	9-10-10	128-bit × 8*	1	127.2



Across 22 workloads, simple CPU model

Figure 2. Performance comparison of DRAM standards

Ramulator Paper and Source Code

- Yoongu Kim, Weikun Yang, and Onur Mutlu,
"Ramulator: A Fast and Extensible DRAM Simulator"
IEEE Computer Architecture Letters (**CAL**), March 2015.
[[Source Code](#)]
- Source code is released under the liberal MIT License
 - <https://github.com/CMU-SAFARI/ramulator>

Ramulator: A Fast and Extensible DRAM Simulator

Yoongu Kim¹ Weikun Yang^{1,2} Onur Mutlu¹
¹Carnegie Mellon University ²Peking University

Ramulator: Free & Open Source

☰ README.md

Ramulator: A DRAM Simulator

Ramulator is a fast and cycle-accurate DRAM simulator [1, 2] that supports a wide array of commercial, as well as academic, DRAM standards:

- DDR3 (2007), DDR4 (2012)
- LPDDR3 (2012), LPDDR4 (2014)
- GDDR5 (2009)
- WIO (2011), WIO2 (2014)
- HBM (2013)
- SALP [3]
- TL-DRAM [4]
- RowClone [5]
- DSARP [6]

The initial release of Ramulator is described in the following paper:

Y. Kim, W. Yang, O. Mutlu. "[Ramulator: A Fast and Extensible DRAM Simulator](#)". In *IEEE Computer Architecture Letters*, March 2015.

For information on new features, along with an extensive memory characterization using Ramulator, please read:

S. Ghose, T. Li, N. Hajinazar, D. Senol Cali, O. Mutlu. "[Demystifying Complex Workload-DRAM Interactions: An Experimental Study](#)". In *Proceedings of the ACM International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*, June 2019 ([slides](#)). In *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, 2019.

[1] Kim et al. *Ramulator: A Fast and Extensible DRAM Simulator*. IEEE CAL 2015.

[2] Ghose et al. *Demystifying Complex Workload-DRAM Interactions: An Experimental Study*. SIGMETRICS 2019.

[3] Kim et al. *A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM*. ISCA 2012.

[4] Lee et al. *Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture*. HPCA 2013.

[5] Seshadri et al. *RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization*. MICRO 2013.

[6] Chang et al. *Improving DRAM Performance by Parallelizing Refreshes with Accesses*. HPCA 2014.

Usage

Ramulator supports three different usage modes.

1. **Memory Trace Driven:** Ramulator directly reads memory traces from a file, and simulates only the DRAM subsystem. Each line in the trace file represents a memory request, with the hexadecimal address followed by 'R' or 'W' for read or write.

- 0x12345680 R
- 0x4cbd56c0 W

• ...

Ramulator: Integration with Other Simulators

☰ README.md

Usage

Ramulator supports three different usage modes.

1. **Memory Trace Driven:** Ramulator directly reads memory traces from a file, and simulates only the DRAM subsystem. Each line in the trace file represents a memory request, with the hexadecimal address followed by 'R' or 'W' for read or write.

- 0x12345680 R
- 0x4cbd56c0 W
- ...

2. **CPU Trace Driven:** Ramulator directly reads instruction traces from a file, and simulates a simplified model of a "core" that generates memory requests to the DRAM subsystem. Each line in the trace file represents a memory request, and can have one of the following two formats.

- `<num-cpuinst> <addr-read>` : For a line with two tokens, the first token represents the number of CPU (i.e., non-memory) instructions before the memory request, and the second token is the decimal address of a *read*.
- `<num-cpuinst> <addr-read> <addr-writeback>` : For a line with three tokens, the third token is the decimal address of the *writeback* request, which is the dirty cache-line eviction caused by the read request before it.

3. **gem5 Driven:** Ramulator runs as part of a full-system simulator (gem5 [7]), from which it receives memory request as they are generated.

For some of the DRAM standards, Ramulator is also capable of reporting power consumption by relying on either VAMPIRE [8] or DRAMPower [9] as the backend.

[7] The gem5 Simulator System.

[8] Ghose et al. *What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study*. SIGMETRICS 2018.

[9] Chandrasekar et al. *DRAMPower: Open-Source DRAM Power & Energy Estimation Tool*. IEEE CAL 2015.

Getting Started

Ramulator requires a C++11 compiler (e.g., `clang++`, `g++-5`).

1. **Memory Trace Driven**

```
$ cd ramulator
$ make -j
$ ./ramulator configs/DDR3-config.cfg --mode=dram dram.trace
Simulation done. Statistics written to DDR3.stats
# NOTE: dram.trace is a very short trace file provided only as an example.
$ ./ramulator configs/DDR3-config.cfg --mode=dram --stats my_output.txt dram.trace
Simulation done. Statistics written to my_output.txt
# NOTE: optional --stats flag changes the statistics output filename
```

2. **CPU Trace Driven**

Ramulator: Reproducibility

Reproducing Results from Paper (Kim et al. [1])

Debugging & Verification (Section 4.1)

For debugging and verification purposes, Ramulator can print the trace of every DRAM command it issues along with their address and timing information. To do so, please turn on the `print_cmd_trace` variable in the configuration file.

Comparison Against Other Simulators (Section 4.2)

For comparing Ramulator against other DRAM simulators, we provide a script that automates the process: `test_ddr3.py`. Before you run this script, however, you must specify the location of their executables and configuration files at designated lines in the script's source code:

- Ramulator
- DRAMSim2 (<https://wiki.umd.edu/DRAMSim2>): `test_ddr3.py` lines 39-40
- USIMM (<http://www.cs.utah.edu/~rajeew/jwac12>): `test_ddr3.py` lines 54-55
- DrSim (<http://ph.ece.utexas.edu/public/Main/DrSim>): `test_ddr3.py` lines 66-67
- NVMain (<http://wiki.nvmain.org>): `test_ddr3.py` lines 78-79

Please refer to their respective websites to download, build, and set-up the other simulators. The simulators must be executed in saturation mode (always filling up the request queues when possible).

All five simulators were configured using the same parameters:

- DDR3-1600K (11-11-11), 1 Channel, 1 Rank, 2Gb x8 chips
- FR-FCFS Scheduling
- Open-Row Policy
- 32/32 Entry Read/Write Queues
- High/Low Watermarks for Write Queue: 28/16

Finally, execute `test_ddr3.py <num-requests>` to start off the simulation. Please make sure that there are no other active processes during simulation to yield accurate measurements of memory usage and CPU time.

Cross-Sectional Study of DRAM Standards (Section 4.3)

Please use the CPU traces (SPEC 2006) provided in the `cputraces` folder to run CPU trace driven simulations.

Other Tips

Power Estimation

For estimating power consumption, Ramulator can record the trace of every DRAM command it issues to a file in DRAMPower [8] format. To do so, please turn on the `record_cmd_trace` variable in the configuration file. The resulting DRAM command trace (e.g., `cmd-trace-chan-N-rank-M.cmdtrace`) should be fed into a compatible DRAM energy simulator such as VAMPIRE [8] or DRAMPower [9] with the correct configuration (standard/speed/organization) to estimate energy/power usage for a single rank (a current limitation of both VAMPIRE and DRAMPower).

Ramulator Already Enabled Many Works

Ramulator: A Fast and Extensible DRAM Simulator

Publisher: IEEE

[Cite This](#)



Yoongu Kim ; Weikun Yang ; Onur Mutlu [All Authors](#)

491

Cites in
Papers

1

Cites in
Patent

6360

Full
Text Views



Abstract

Abstract:

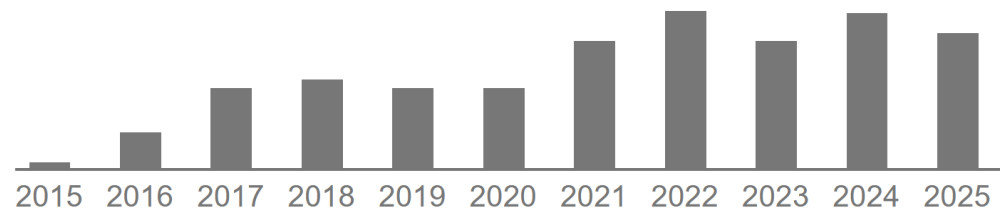
Recently, both industry and academia have proposed many different roadmaps for the future of DRAM. Consequently, there is a growing need for an extensible DRAM simulator, which can be easily modified to judge the impact of different DRAM technologies on system performance. Ramulator is a fast and extensible DRAM simulator that can be easily modified to judge the impact of different DRAM technologies on system performance. Unlike existing DRAM simulators, Ramulator is designed to be easily modified to judge the impact of different DRAM technologies on system performance.

Document Sections

1 Introduction

2 Ramulator: High-Level

Total citations Cited by 957



Ramulator 2.0

A Modern, Modular, and Extensible
DRAM Simulator

[IEEE Computer Architecture Letters'24]

Motivation

- **Robustness issues** in DRAM
 - Data retention
 - Read disturbance (RowHammer, RowPress, etc.)
 - ...
- **Performance issues** in main memory system
 - Performance overhead analysis of read disturbance mitigation techniques
 - Processing-in-Memory architectures
 - Emerging memory technologies
 - ...

DRAM simulation infrastructures are needed
to understand, characterize, and evaluate
the robustness and performance of DRAM

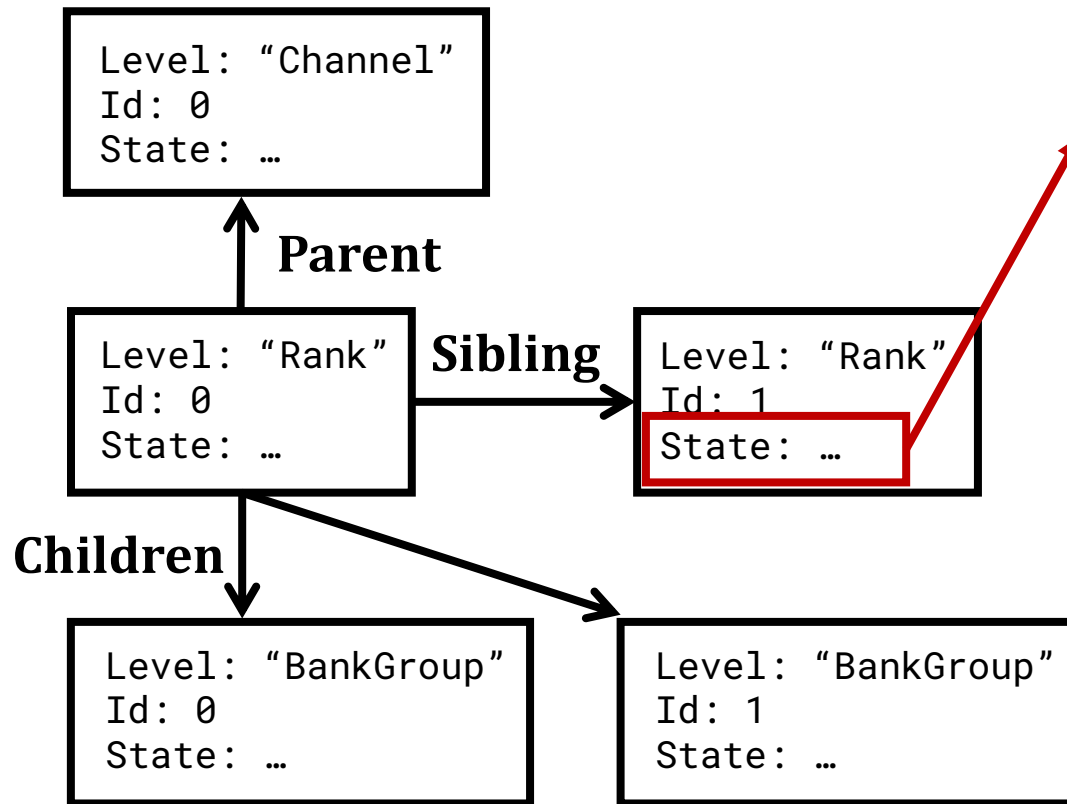
Executive Summary

Ramulator 2.0: Modern, modular, and extensible
DRAM & memory system simulator

- Models a wide range of **DRAM standards** and **memory controller functionalities**
- **Modular and extensible** software architecture
- **Fine-grained modeling** of DRAM operation (cycle-level)
- Used in a wide range of research works

Ramulator 2.0 Design and Features (I)

- **Hierarchical state-machine** based modeling of DRAM



State of a DRAM node:

- **Current state:** open, close, activating, etc.
- **Timing constraints:** Earliest time in the future that each DRAM command is allowed to be issued
- **Energy & power:** Time spent in each state and the number of DRAM commands served (DRAMPower model)
- **Can be extended to include more**

Ramulator 2.0 Design and Features (II)

- DRAM commands implemented as lambda functions that **hierarchically traverses and updates the states** of the nodes
 1. Checks the current states of the nodes to **decode which DRAM command to issue**
 2. **Programmatically** apply state changes
 3. Updates the **timing constraints, power metrics**, etc.
- **Templated** library of generalized DRAM command lambda functions allow **reuse of command implementations** across different DRAM standards

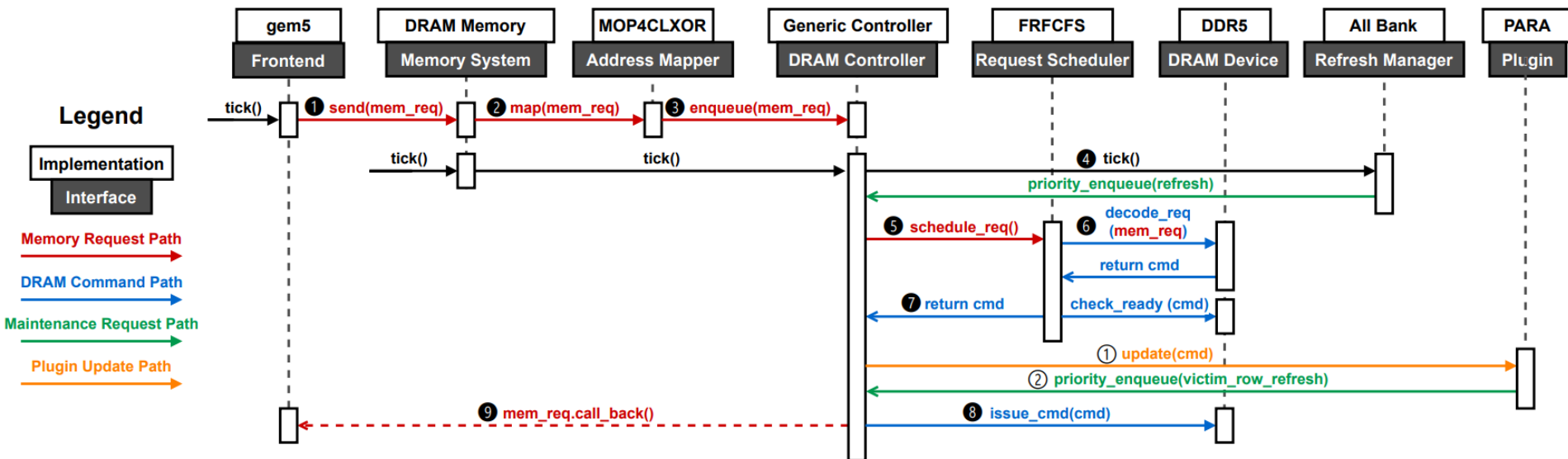
```
template <class DRAM_t>
int RequireAllBanksClosed(typename DRAM_t::Node* node,
    int cmd, int target_id, Clk_t clk) {
    // for all banks {
    // ...
    if (bank->m_state == DRAM_t::m_states["Closed"]) {
        continue;
    } else {
        return T::m_commands["PREab"];
    }
    // }
    return cmd;
};
```

Applicable to:
DDR3, DDR4, DDR5
LPDDR4, LPDDR5
HBM (1/2/3), GDDR6

Example DRAM Command Decode Function

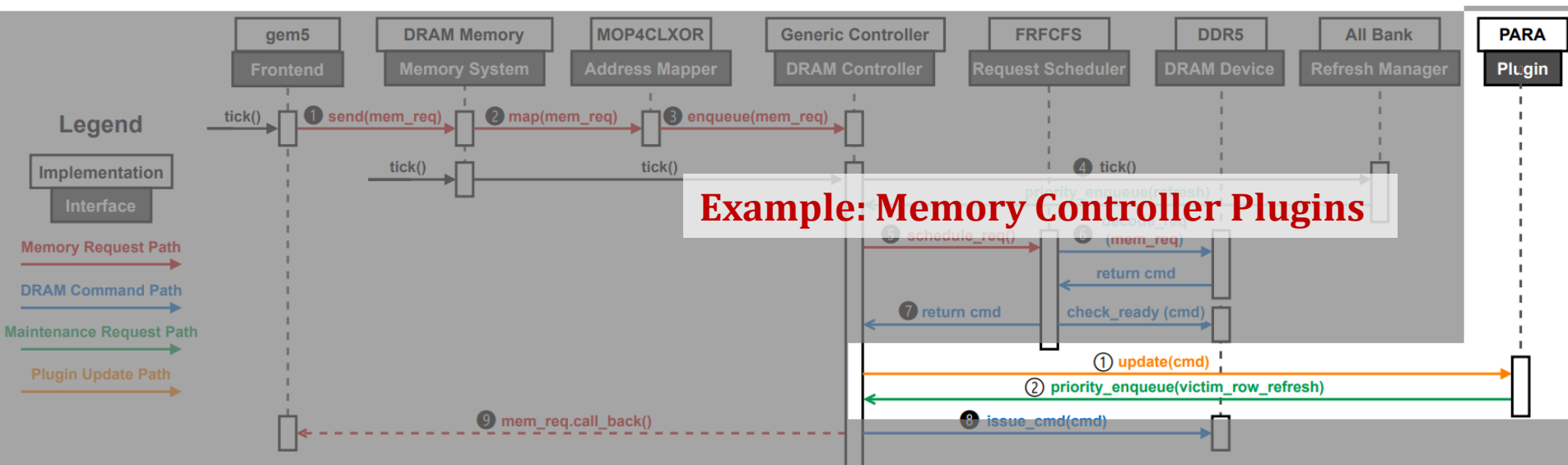
Ramulator 2.0 Design and Features (III)

- Modular and extensible software architecture
 - All components in the memory system modeled with the **same interface** and **different implementations**
 - Example: The memory controller include:
 - Address Mapper, Request Scheduler, Refresh Controller, Row Policy, etc.
 - Each can be **flexibly changed** without hardcoding other parts



Ramulator 2.0 Design and Features (IV)

- Modular and extensible software architecture
 - All components in the memory system modeled with the **same interface** and **different implementations**
 - Example: The memory controller include:
 - Address Mapper, Request Scheduler, Refresh Controller, Row Policy, etc.
 - Each can be **flexibly changed** without hardcoding other parts



Ramulator 2.0 Design and Features (V)

- More in the paper
 - More detailed explanation of modeling methodology
 - Authoring of DRAM specifications (organization, timings, etc.)
 - Memory controller plugin & RowHammer mitigations
 - Performance comparison with other DRAM simulators

Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator

Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, and Onur Mutlu



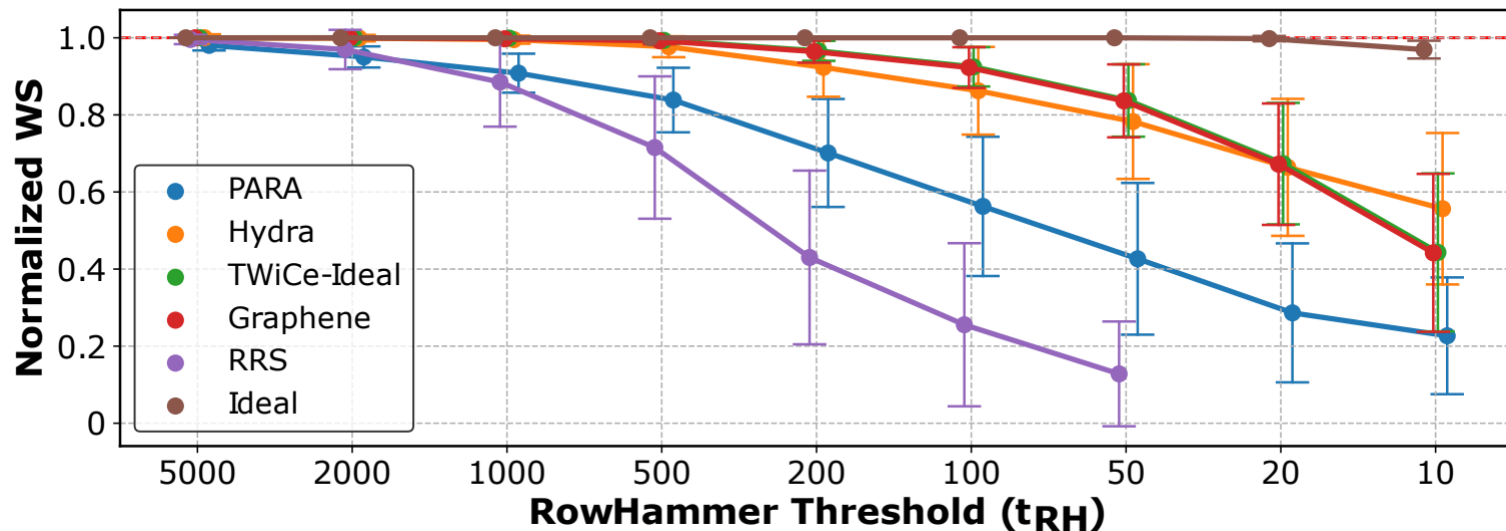
IEEE CAL Paper



Open-source version Github repo:
CMU-SAFARI/ramulator2

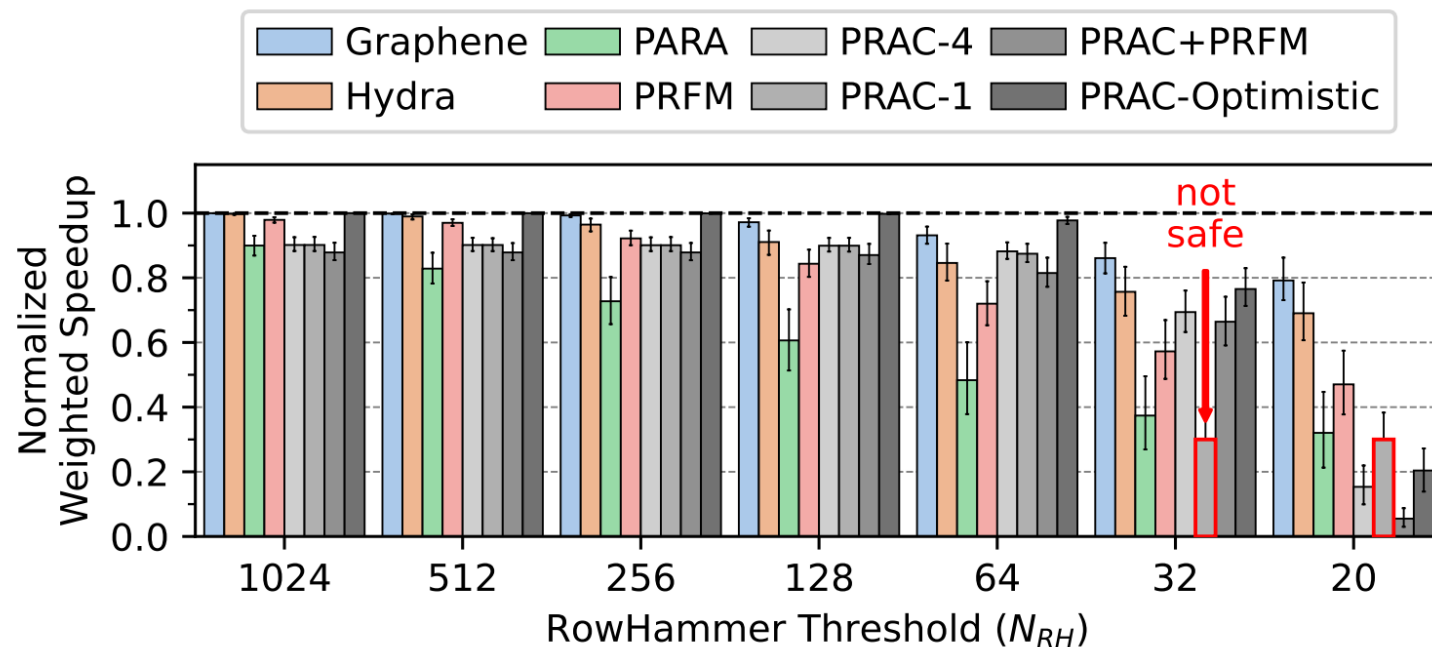
Ramulator 2.0 Case Studies (I)

- **Cross-section performance overhead evaluation of different RowHammer mitigation techniques**
[Luo+, IEEE CAL 2023]
 - Six different RowHammer mitigation techniques all implemented **as plugins to the same memory controller implementation**



Ramulator 2.0 Case Studies (II)

- Performance evaluation of DDR5 Per Row Activation Counting (PRAC) [Canpolat+, DRAMSec'24]
 - Memory controller implementation extended with support for per-row activation count tracking and back-off signal



Ramulator 2.0: Validation (I)

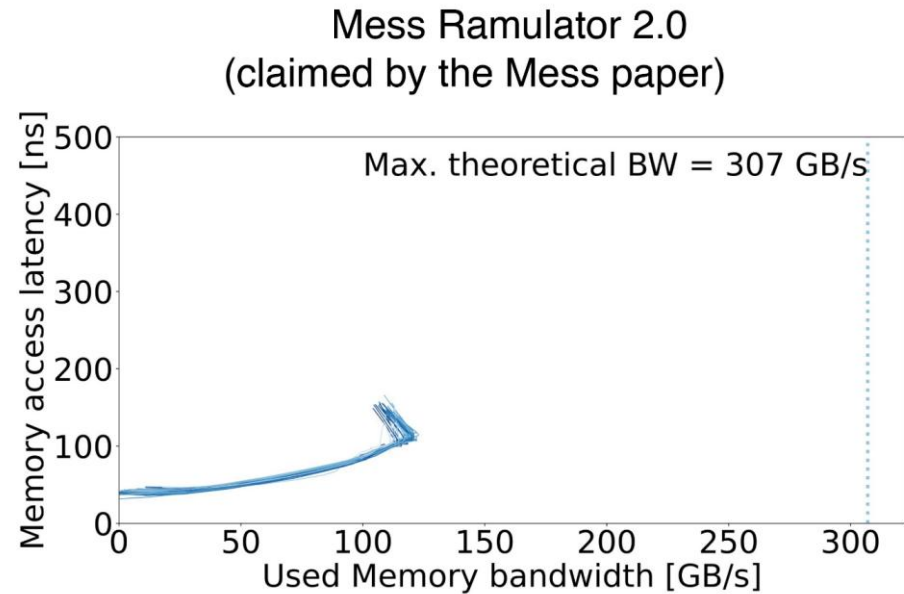
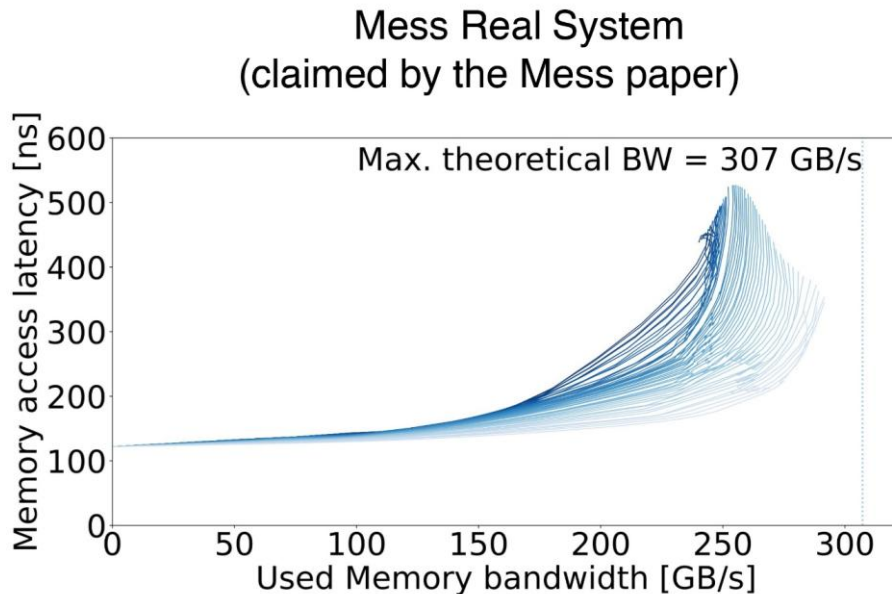
- Ramulator 2.0's DRAM command traces are validated against Micron's DDR4 Verilog model [1]
 - Across streaming- and random-access patterns with varying memory intensities

We find no timing or state transition violations

Ramulator 2.0:

Comparison Against Real Systems (I)

- A recent work aims to compare the accuracy of memory simulators including Ramulator against real systems
 - Using latency-bandwidth curves
 - Across varying read/write intensities

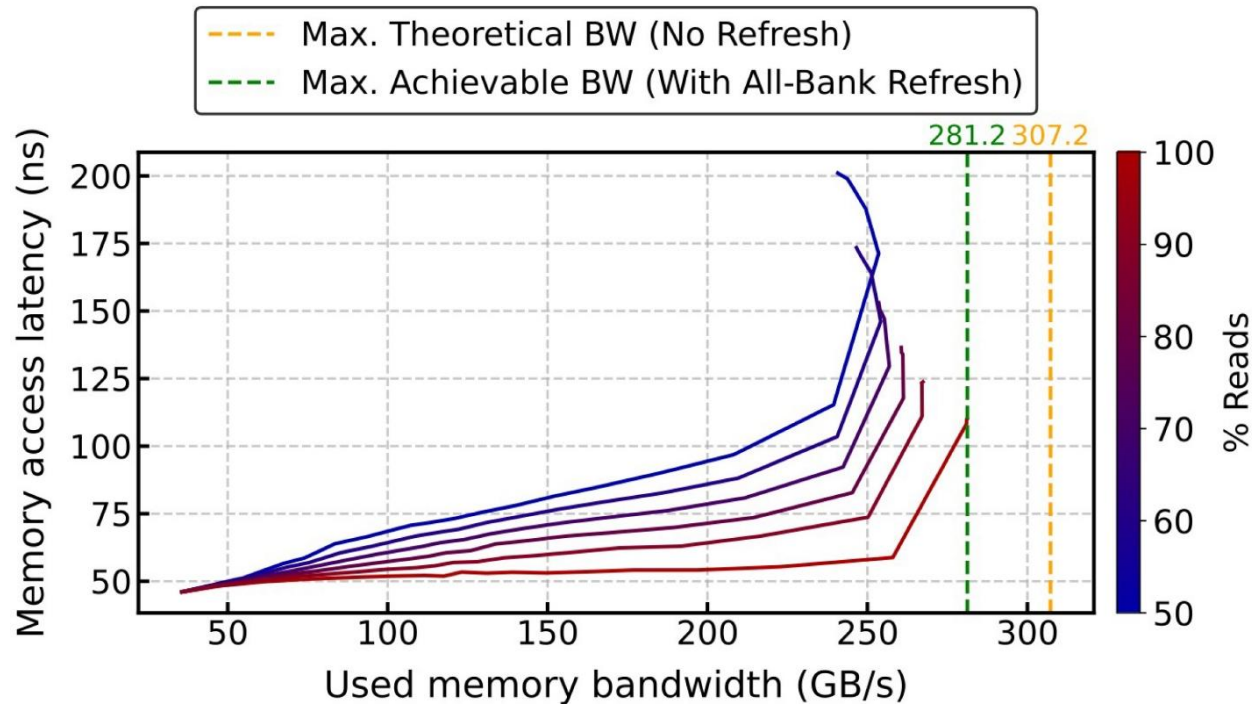


Results claimed by the Mess paper poorly resembles the real system results due to their major configuration errors

Ramulator 2.0:

Comparison Against Real Systems (II)

- We identified multiple trivial human errors in configuration of Ramulator 2.0 in the Mess paper



When correctly configured, Ramulator 2.0's simulated memory system performance actually resembles real system characteristics

Ramulator 2.0: Comparison Against Real Systems (III)

Cleaning up the Mess

Haocong Luo Ataberk Olgun Maria Makeenkova F. Nisa Bostancı
Geraldo F. de Oliveira A. Giray Yağlıkçı Onur Mutlu

ETH Zürich

A MICRO 2024 best paper runner-up publication (the Mess paper [1]) with all three artifact badges awarded (including “Reproducible”) proposes a new benchmark to evaluate real and simulated memory system performance. The publication contends that Ramulator 2.0 [2] and DAMOV [3] (zsim+Ramulator) (along with other existing memory system simulators) “poorly resemble the actual system performance” and asserts that their simulator is better.

In this paper, we demonstrate that the Ramulator 2.0 simulation results reported in [1] are incorrect and, at the time of the publication of [1], irreproducible. We find that the authors of [1] made multiple trivial human errors in both the configuration and usage of the simulators. We show that by correctly configuring Ramulator 2.0, Ramulator 2.0’s simulated memory system performance actually resembles real system characteristics well, and thus a key claimed contribution of [1] is factually incorrect. We also identify that the DAMOV simulation results in [1] use wrong simulation statistics that are unrelated to the simulated DRAM

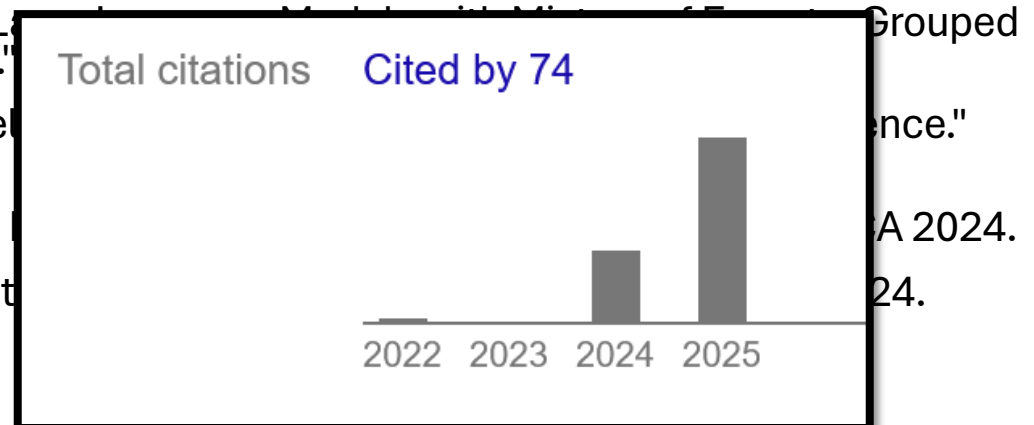
A MICRO 2024 best paper runner-up publication, A Mess of Memory System Benchmarking, Simulation and Application Profiling [1], which we refer to as “the Mess paper” throughout this paper, with all three artifact badges awarded (including “Reproducible”) proposes a new benchmark to evaluate real and simulated memory system performance. While doing so, it makes some strong negative claims about Ramulator 2.0 and DAMOV [3, 9], which we demonstrate to be incorrect and are due to configuration errors made in the Mess paper.

Fig. 1-(a) and (b) (copied from Fig. 6 and Fig. 4 in [1]) show average memory latency as a function of throughput (used memory bandwidth) for the Mess benchmark using a Ramulator 2.0 simulation and a real system, respectively, as depicted in [1].¹ From these two figures, the authors of the Mess paper conclude that Ramulator 2.0 “poorly resembles the actual system performance” (Section I of [1]). They write “The simulated memory latency is unrealistically low and the maximum simulated memory bandwidth is only 126 GB/s which is less

<https://arxiv.org/pdf/2510.15744>

Ramulator 2.0 Already Enabled Many Works

- Yuksel, Ismail, et al. "PuDHammer: Experimental Analysis of Read Disturbance Effects of Processing-using-DRAM in Real DRAM Chips" ISCA 2025.
- Li, Cong, et al. "H2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference." ISCA 2025.
- Canpolat, Oğuzhan, et al. "Chronus: Understanding and securing the cutting-edge industry solutions to dram read disturbance." HPCA 2025.
- Olgun, Ataberk, et al. "Variable Read Disturbance: An Experimental Analysis of Temporal Variation in DRAM Read Disturbance." HPCA 2025.
- Woo, Jeonghyun, et al. "QPRAC: Towards Secure and Practical PRAC-based RowHammer Mitigation using Priority Queues." HPCA 2025.
- Woo, Jeonghyun, and Prashant J. Nair. "DAPPER: A Performance-Attack-Resilient Tracker for RowHammer Defense." HPCA 2025.
- Wang, Cheng, and Mingyu Gao. "UniZK: Accelerating Zero-Knowledge Proof with Unified Hardware and Flexible Kernel Mapping." ASPLOS 2025.
- Yun, Sungmin, et al. "Duplex: A Device for L Query Attention, and Continuous Batching." HPCA 2025.
- Lee, Seung Yul, et al. "VGA: Hardware Acceleration for RowHammer Mitigation." MICRO 2024.
- Zhao, Yilong, et al. "UM-PIM: DRAM-based PIM Accelerator for Matrix Multiplication." HPCA 2024.
- Li, Jinhao, et al. "MARCA: Mamba Accelerator for Matrix Multiplication." HPCA 2024.
- And many more other works...



Conclusion & Future Work

Ramulator 2.0: Modern, modular, and extensible
DRAM & memory system simulator

- Fine-grained modeling of DRAM operation (cycle-level)
- Models a wide range of DRAM standards and memory controller functionalities
- Used in a wide range of research works

Ongoing & Future Works

- Unit & regression test coverage
- More DRAM standards and emerging technologies
- More detailed memory controller modeling (i.e., pipelined scheduler and gear ratio)
- Generalizable modeling for PuM/PnM architectures
- ...

Conclusion & Future Work

112

IEEE COMPUTER ARCHITECTURE LETTERS, VOL. 23, NO. 1, JANUARY-JUNE 2024

Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator

Haocong Luo , Yahya Can Tuğrul , F. Nisa Bostancı , Ataberk Olgun , *Graduate Student Member, IEEE*,
A. Giray Yağlıkçı , *Graduate Student Member, IEEE*, and Onur Mutlu , *Fellow, IEEE*

Abstract—We present Ramulator 2.0, a highly modular and extensible DRAM simulator that enables rapid and agile implementation and evaluation of design changes in the memory controller and DRAM to meet the increasing research effort in improving the performance, security, and reliability of memory systems. Ramulator 2.0 abstracts and models key components in a DRAM-based memory system and their interactions into shared *interfaces* and independent *implementations*. Doing so enables easy modification and extension of the modeled functions of the memory controller and DRAM in Ramulator 2.0. The DRAM specification syntax of Ramulator 2.0 is concise and human-readable, facilitating easy

parameter changes) in the memory controller and DRAM. Unfortunately, existing cycle-accurate DRAM simulators are not modular and extensible *enough* to meet such a requirement.

We identify two key issues in the design and implementation of existing cycle-accurate DRAM simulators. First, they do *not* model key components of a DRAM-based memory system in a *fundamentally modular* way, making it difficult to implement and maintain different intrusive design changes. For example, USIMM [1] does not separate the DRAM specification from the memory controller. Similarly, the templated implementations

We are happy to support
and help anyone who wants to use it



Paper



GitHub

Ramulator: A Brief Introduction

Presenter:
Nisa Bostancı
Prof. Onur Mutlu
XiangShan Tutorial
@ MICRO'25